Assignment 1: Banking Accounts System (Inheritance + Overriding)

Domain: Banking

Problem Statement

Build a mini banking system where different account types follow different rules.

Classes to Create

1. `Account` (Base class)
2. `SavingsAccount` (Child class)
3. `CurrentAccount` (Child class)

Requirements

Base Class: `Account`

- Fields: `accountNumber`, `holderName`, `balance`
- Constructor to initialize fields
- Methods:
  - `deposit(double amount)`
  - `withdraw(double amount)` *(base version: basic withdrawal)*
  - `displayDetails()`

Child Class: `SavingsAccount`

- Extra field: `minBalance` (ex: 1000)
- Override `withdraw()`:
  - Withdrawal not allowed if balance goes below minBalance
- Use `super.withdraw()` only when allowed

Child Class: `CurrentAccount`

- Extra field: `overdraftLimit`
- Override `withdraw()`:
  - Allow withdrawal even if balance becomes negative, but only up to overdraftLimit

Additional Tasks

- Create a `BankApp` main program:
  - Create one Savings and one Current account
  - Perform deposits and withdrawals

    - Print output after each action

First-principle focus

- “What rule changes between types?”
- “What stays common and should remain in base class?”

Assignment 2: Employee Payroll System (Inheritance + Polymorphism)

Domain: HR / Payroll

Problem Statement

Create a payroll system where different employee types calculate salary differently.

Classes to Create

1. `Employee` (Base class)
2. `FullTimeEmployee` (Child)
3. `PartTimeEmployee` (Child)
4. `ContractEmployee` (Child)

Requirements

Base Class: `Employee`

- Fields: `empId`, `name`
- Method: `calculateSalary()` (default/basic version or abstract—your choice)
- Method: `displayEmployee()`

Child Classes Override calculateSalary()

- `FullTimeEmployee`: salary = basic + hra + da
- `PartTimeEmployee`: salary = hoursWorked * hourlyRate
- `ContractEmployee`: salary = fixedAmount - tax

Additional Tasks (Moderate Level)

- In `Main`, store different employees in **Employee[] array**
- Loop and call `calculateSalary()` for each
- Print payslip for each employee

First-principle focus

- "Same message, different behavior" (polymorphism)
- Avoid if-else by using overriding

Assignment 3: Notification System (Inheritance + super + Constructor Chaining)

Domain: Communication / Alerts

Problem Statement

Build a notification system where each type of notification has a different way of sending messages.

Classes to Create

1. `Notification` (Base class)
2. `EmailNotification` (Child)
3. `SMSNotification` (Child)
4. `PushNotification` (Child)

Requirements

Base Class: `Notification`

- Fields: `recipient`, `message`
- Constructor: initialize fields
- Method:
  - `send()` *(default implementation prints "Sending notification…")*

Child classes

Override `send()`:

- `EmailNotification`: print "Email sent to …"
- `SMSNotification`: print "SMS sent to …"
- `PushNotification`: print "Push alert sent to …"

Moderate Level Add-ons

- Add **logging** common behavior in base class:
  - `logNotification()` method in base class
- Each child must call `super.logNotification()` before sending.

First-principle focus

- Common behavior should be in parent class
- Specialized behavior should be in child classes